Análise e Projeto Orientado a Objetos

Introdução

Prof. Fellipe Aleixo (*fellipe.aleixo@ifrn.edu.br*)

Objetivos da Disciplina

- **Aprender** e **praticar** a análise e projeto de sistemas orientados a objetos:
  - Compreensão das necessidades
  - Análise do problema
  - Projeto da solução

- Contextualizar as práticas de APOO em um processo de desenvolvimento de um **sistema de informação**

Sistema de Informação

- Um SI é uma combinação de pessoas, dados, processos, interfaces, redes de comunicação e tecnologia que interagem com o objetivo de dar suporte e melhorar o **processo de negócio** de uma organização com relação às informações
  - Vantagens do ponto de vista competitivo

- Objetivo principal e final da construção de um SI: adição de valor à organização

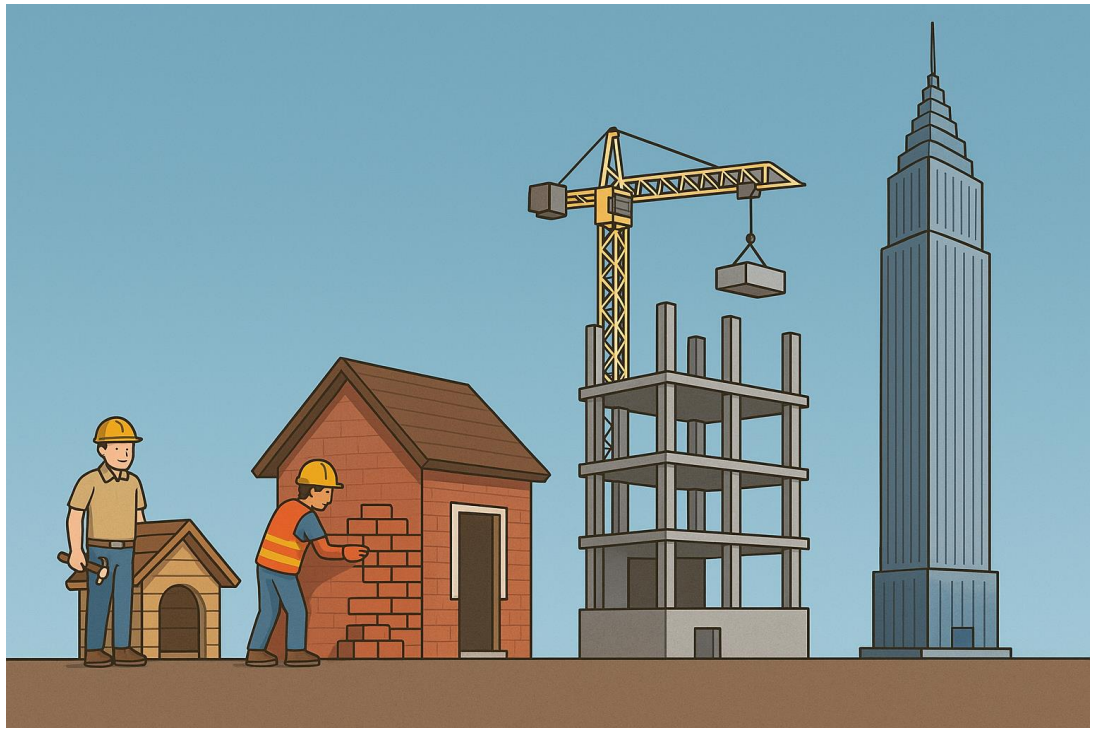
Sistema de Software

Um dos componentes de um SI é denominado sistema de software

Compreende os módulos funcionais computadorizados que interagem entre si para proporcionar a automatização de diversas tarefas

Característica intrínseca do desenvolvimento de sistemas de software: **complexidade**

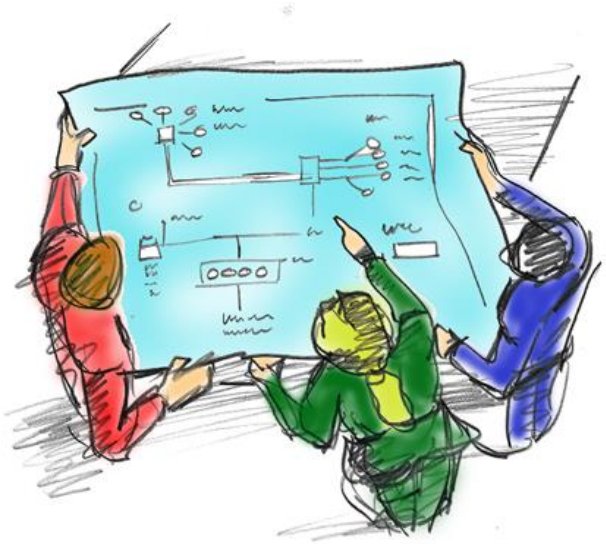
Sistemas de Software



AUMENTO DA COMPLEXIDADE

Modelos de Software

- Na construção de sistemas de software, assim como na construção de sistemas habitacionais, também há uma gradação de complexidade
  - Há sistemas que **necessitam** serem projetados antes



Modelos de Software

- Um modelo pode ser visto como uma representação idealizada de um sistema que se planeja construir

- **Maquetes** de edifícios e de aviões e **plantas** de circuitos eletrônicos são apenas alguns exemplos de modelos

Razões para Construção de Modelos

A princípio, podemos ver a construção de modelos como uma atividade que atrasa o desenvolvimento do software propriamente dito

Mas essa atividade propicia...

- O gerenciamento da **complexidade** inerente ao desenvolvimento de software
- A comunicação entre as pessoas envolvidas
- A redução dos custos no desenvolvimento
- A predição do comportamento futuro do sistema

Entretanto, note o fator complexidade como condicionante dessas vantagens

Fluxos de Atividades

Uma introdução a um processo de desenvolvimento de software

Processo Unificado

- Termo em inglês: ***Unified Process*** ou simplesmente **UP**

- O Processo Unificado é um processo bastante conciso e eficiente para análise e projeto de sistemas de software

- Neste método, cada artefato (documento ou diagrama) tem uma razão muito clara para existir e as conexões entre os diferentes artefatos são muito precisas

Desenvolvimento de Sistemas OO

- Por que usar orientação a objetos?
- Basta usar uma linguagem orientada a objetos?
- Há vantagens em mudar o processo de desenvolvimento de sistemas das empresas?

Questão da Ferramenta

- Comprar um martelo não transforma você em um arquiteto!

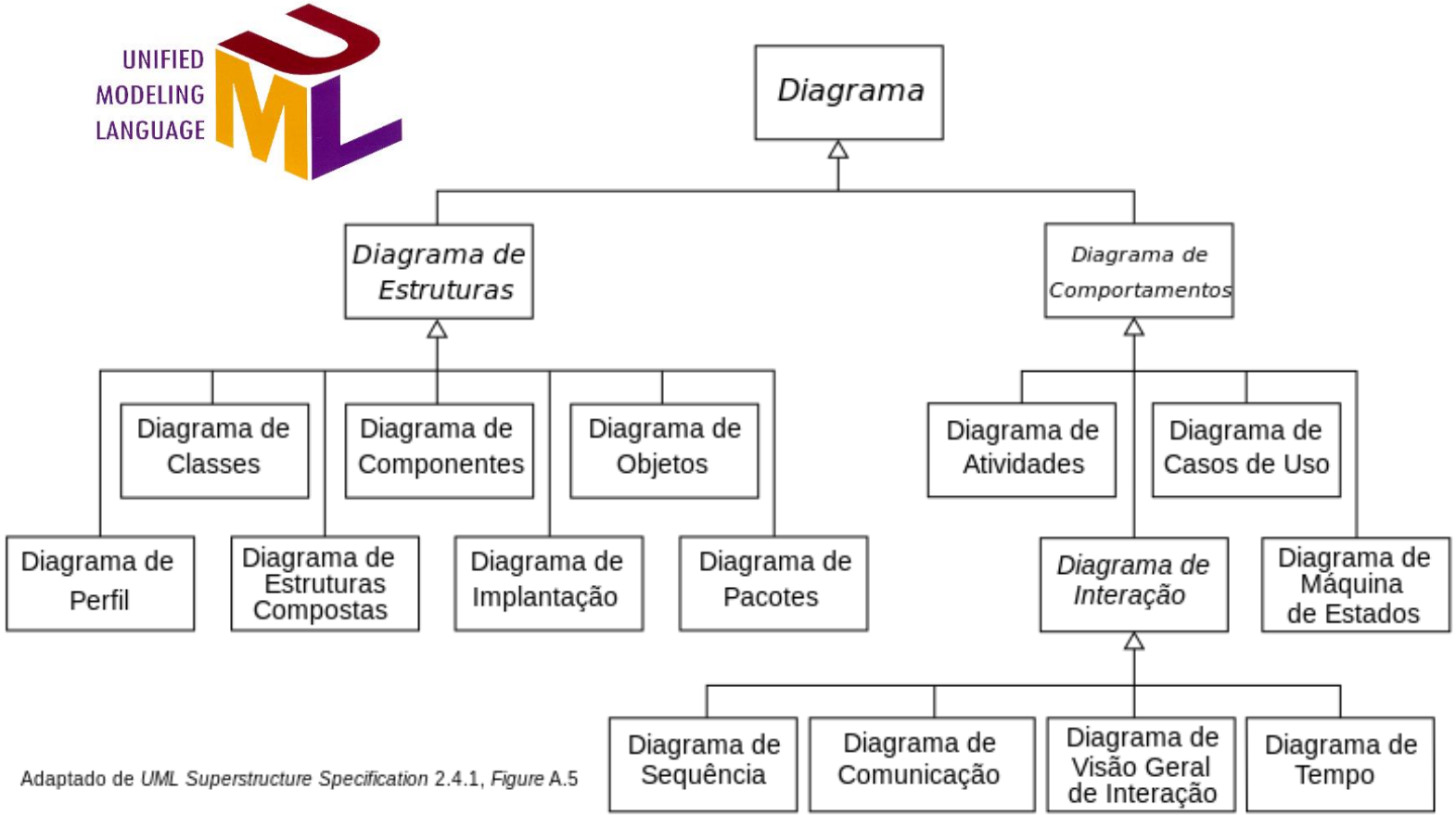
UML

- ***Unified Modeling Language***

- Conhecer uma linguagem não implica na habilidade de saber usá-la para produzir artefatos úteis

- Escrever bons projetos é como escrever poesia
  - Não basta conhecer a linguagem, é preciso dominar certas técnicas de escrita

Diagramas da UML



Adaptado de *UML Superstructure Specification* 2.4.1, *Figure* A.5

Software Deselegante

- O software deselegante é aquele software feito sem uma estrutura clara
- O software deselegante é aquele do qual não se consegue reusar partes e que não se consegue entender como funciona sem uma boa carga de documentação (e muitas vezes nem assim)
- É também aquele no qual uma pequena modificação em uma de suas características pode causar um não funcionamento generalizado

Software Elegante

- O software elegante é o software cuja estrutura é intrinsecamente mais fácil de compreender, que é auto documentado e pode ser compreendido em nível macro ou em detalhes

- Ele é mais fácil de modificar: quando alguma de suas características é mudada, ele continua funcionando

Soluções para Prover Elegância

- ***Design Patterns***, ou **Padrões de Projeto**, que espelham lições aprendidas ao longo dos anos em diferentes projetos

- Soluções documentadas para problemas específicos e recorrentes

Atividades do Desenvolvimento

1. Análise
2. Projeto
3. Implementação
4. Teste

Análise

- A análise enfatiza a investigação do **problema**
- O objetivo da análise é levar o analista a investigar e a descobrir
- Para que esta etapa seja realizada em menos tempo e de forma mais precisa, deve-se ter um bom método de trabalho

Análise

- Pode-se dizer que o resultado da análise é o enunciado do problema, e que o projeto será a sua resolução

- Problemas mal enunciados podem até ser resolvidos, mas a solução não corresponderá às expectativas

Análise

- A qualidade do processo de análise é importante porque um erro de concepção
  - resolvido na fase de análise tem um custo
  - resolvido na fase de projeto tem um custo maior
  - resolvido na fase de implementação maior ainda
  - resolvido na fase de implantação do sistema tem um custo relativamente astronômico

Projeto

- A fase de projeto enfatiza uma **proposta de solução** que atenda os requisitos da análise

- Então, se a analise é uma investigação para tentar descobrir o que o cliente quer, o projeto consiste em propor uma solução com base no conhecimento adquirido na análise

Implementação

- A utilização de técnicas sistemáticas nas fases de análise e projeto faz com que o processo de geração de código possa ser automatizado

- Neste caso, cabe ao programador dominar as características específicas das linguagens, ferramentas, frameworks e estruturas de dados para adaptar o código gerado aos requisitos indicados quando necessário

Testes

- A fase de testes envolve, por exemplo:
  - **Testes de unidade** - realizados pelo programador, para verificar se os componentes implementados atendem à especificação do projetista
  - **Testes funcionais** - normalmente efetuados por um analista experiente, e visam verificar a adequação do sistema aos requisitos inicialmente levantados

As Fases do Processo Unificado

- A fase de **concepção** incorpora o estudo de viabilidade e uma parte da análise de requisitos
- A fase de **elaboração** incorpora a maior parte da análise de requisitos, a análise de domínio e o projeto e validação da arquitetura do sistema
- A fase de **construção** corresponde à programação e testes de todas as funcionalidades previstas
- A fase de **transição** consiste na instalação no ambiente do cliente e manutenção do mesmo

Concepção

- Análise de Requisitos
- Escopo do Sistema
- Modelagem dos Casos de Uso
- Esboço de Modelo Conceitual
- Cronograma de Desenvolvimento e Desembolso

Análise de Requisitos

- A análise de requisitos é fundamental para o desenvolvimento de sistemas, pois trata justamente de descobrir o que o cliente quer com o sistema

- A análise de requisitos está associada ao processo de descobrir quais são as operações que o sistema deve realizar e quais são as restrições que existem sobre estas operações

Requisitos

- **Funcionais** – o que o sistema deve fazer
- **Não-funcionais** – restrições sobre como o sistema deve desempenhar suas funções

Exemplos - Requisitos Funcionais (RF)

“O que o sistema DEVE fazer – funcionalidades específicas”

- **RF-001 – Autenticação de usuários**
  - O sistema deve permitir login com e-mail e senha.
  - Deve oferecer recuperação de senha via e-mail.
- **RF-002 – Cadastro de produtos**
  - O sistema deve permitir que administradores cadastrem novos produtos, incluindo nome, descrição, preço e imagem.
- **RF-003 – Processamento de pedidos**
  - O sistema deve calcular o valor total do pedido (incluindo impostos e frete).
  - Deve gerar um comprovante em PDF após a finalização.
- **RF-004 – Busca avançada**
  - Os usuários devem poder filtrar produtos por categoria, preço e avaliação.

Exemplos - Requisitos Não Funcionais (RNF)

"Critérios de QUALIDADE, DESEMPENHO ou RESTRIÇÕES do sistema"

- **RNF-001 – Performance**
  - O sistema deve responder a 90% das requisições em menos de 2 segundos.
- **RNF-002 – Segurança**
  - Dados sensíveis (como senhas) devem ser criptografados (ex.: SHA-256).
  - O sistema deve bloquear tentativas de login após 5 falhas consecutivas.
- **RNF-003 – Compatibilidade**
  - Deve funcionar nos navegadores Chrome, Firefox e Edge (versões atualizadas).
  - Deve ser responsivo em dispositivos móveis (Android/iOS).
- **RNF-004 – Escalabilidade**
  - O sistema deve suportar até 1.000 usuários simultâneos sem degradação de performance.
- **RNF-005 – Disponibilidade**
  - O sistema deve ter uptime de 99,9% (ex.: tolerância a falhas de até 8h/ano).

Entrevista – Levantamento de Requisitos

- **<u>Perguntas Gerais sobre o Projeto</u>**
  1. Qual é o objetivo principal do sistema?
  2. Quem são os usuários finais (personas)?
  3. Existe um sistema legado que será substituído? Quais são seus problemas atuais?
  4. Quais métricas de sucesso serão usadas (ex.: tempo de resposta, número de usuários simultâneos)?

Entrevista – Levantamento de Requisitos

- **Requisitos Funcionais** (O QUE o sistema deve fazer)
  1. Quais são as funcionalidades essenciais? (Ex.: cadastro de usuários, checkout de pedidos)
  2. Quais regras de negócio devem ser aplicadas? (Ex.: "Desconto de 10% para compras acima de R$ 100")
  3. Como o sistema deve lidar com casos de erro? (Ex.: e-mail já cadastrado, estoque insuficiente)
  4. Quais integrações externas são necessárias? (Ex.: API de pagamento, ERP, SMTP)

Entrevista – Levantamento de Requisitos

- **Requisitos Não-Funcionais** (COMO o sistema deve funcionar)
  1. Qual é o volume esperado de usuários/dados? (Ex.: 1.000 req/segundo)
  2. Quais são os requisitos de segurança? (Ex.: autenticação com 2FA, criptografia de dados)
  3. Qual é a tolerância a falhas? (Ex.: sistema deve estar 99,9% disponível)
  4. Há restrições tecnológicas? (Ex.: "Deve rodar em Docker")

Erro Comum

- Deve ficar claro ao analista que requisitos são coisas que o **cliente** ou **usuário** solicitam, e não coisas que ele, como analista, idealizou